

P1886R0

Error speed benchmarking

Published Proposal, 2019-10-05

This version:

<https://wg21.link/P1886R0>

Author:

[Ben Craig](#) (National Instruments)

Audience:

WG21; DG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

[github](#)

Abstract

The author measures the speed of various error handling approaches. Abort and x64 exceptions are fast on the happy path. Error code strategies are fast on the sad path.

Table of Contents

- 1 Introduction**
- 2 Measuring methodology**
- 3 Justification of test case choices**
- 4 Starter test case for on-line analysis**
- 5 Happy path measurements**
 - 5.1 Write to a global
 - 5.2 Write through an argument
 - 5.3 "Return" a value
- 6 Sad path measurements**
 - 6.1 Write to a global
 - 6.2 Write through an argument
 - 6.3 "Return" a value
- 7 Off-line analysis**
 - 7.1 LLVM-MCA happy path off-line analysis
 - 7.2 Sad path affecting the happy path and macro-benchmark theorizing
- 8 Conclusion**
- 9 Acknowledgments**

Appendix A: Additional error handling types

Write to a global (happy path)

Happy path: Write through an argument

Happy path: "Return" a value

Sad path: Write to a global

Sad path: Write through an argument

Sad path: "Return" a value

Appendix B: The build flags

MSVC

Clang x64

GCC x64

Appendix C: The code

Common support code

Write to a global

Write through an argument

"Return" a value

Appendix D: Histograms

Happy path: Write to a global
Happy path: Write through an argument
Happy path: "Return" a value
Exception sad path: Write to a global
Exception sad path: Write through an argument
Exception sad path: "Return" a value
Other sad path: Write to a global
Other sad path: Write through an argument
Other sad path: "Return" a value

References

Informative References

§ 1. Introduction

In this paper, the relative speeds of error handling strategies in micro-benchmarking scenarios will be analyzed. "Happy" and "sad" path timings will be presented.

§ 2. Measuring methodology

All benchmarks lie. It's important to know how a benchmark is set up so that the useful parts can be distinguished from the misleading parts.

Gathering representative timings for very small pieces of code is difficult on modern hardware. One type of micro-architectural quirk that makes these measurements very painful is code alignment [\[Bakhvalov\]](#). Each call and each branch can introduce stalls if jumping to a poorly aligned location. These stalls can dominate the timings. The author has seen code alignment differences swing a benchmark running at 451 cycles per iteration to 310 cycles per iteration, for a 45% increase in execution time. This involved no system calls, no atomic operations, no divisions, or any other highly expensive operations being performed.

This research used a novel technique to get representative timings across a wide range of jump alignments. At the beginning of each function, a random number of `nop` instructions (0 to 31 inclusive) were inserted. A matching set of `nop` instructions were inserted at a later point in the execution path so that a total of 31 user inserted `nop` instructions were always executed. 1024 instances of this program, all with different layouts of nops, were then built and run.

Tests were compiled to optimize for speed. Link-time optimizations (LTO) / whole program optimizations were not used. Had LTO been used, the entire program would be optimized away, and the benchmarks would be invalid. Some compilers only support profile-guided optimizations (PGO) after turning on link-time optimizations, so PGO wasn't used either. Only the exception cases were built with exceptions turned on.

For all but the exception failure cases, 16K warm-up iterations were run before starting timing. Within the timer bounds, 512K iterations of the test case were run. The exception failure cases only had 256 warm-up iterations and 16K timed iterations, due to how slow throwing an exception can be.

These tests are all testing code that is "hot". Before the tests are timed, the scenario under test is run many iterations to "warm up" the code. This should ensure that the code is all in cache, and leave the branch predictors in the ideal state. Cold code benchmarks would likely show different results.

All tests were run on the same machine. The processor was an Intel Core i7-3770 running at 3.3915 GHz. The Core i7-3770 has an Ivy Bridge micro-architecture, and was released in April 2012. C states, HyperThreading, Intel TurboBoost and Intel SpeedStep were all disabled in order to improve benchmark reproducibility. MSVC benchmarks were run on the 64-bit

version of Windows 10 Enterprise (10.0.17134.1006). Clang and GCC benchmarks were run on Linux slax 4.9.0-9-amd64 #1 SMP Debian 4.9.168-1 (2019-04-12) x86_64 GNU/Linux.

§ 3. Justification of test case choices

This paper takes three abstract programs, translates them into several different error handling strategies, then benchmarks roughly a thousand nop-randomized versions of those programs, all on one machine. Ideally, there would be many machines of different architectures and micro-architectures used. There would be dozens, or hundreds of abstract programs chosen, and all performance differences would be root caused. The results would then be run through rigorous statistical devices, correlating the results of the abstract programs and giving confidence values.

The author doesn't have the time, resources, or expertise for that. The author (arguably) has the time, resources, and expertise to benchmark more than just one abstract program.

These programs perform virtually no "real" work. In many production scenarios, the costs of error handling will be hidden through out-of-order execution in combination with expensive operations like I/O, synchronization, memory operations, and "large" math operations. In a production environment, it is possible that the error handling overhead differences only manifest in subtle, hard to observe ways.

These programs are crafted in such a way as to make it possible to measure error overhead. To use an analogy, it is difficult to measure the weight of a flea. It is even more difficult to measure the weight of a flea on an elephant by subtracting out the weight of the elephant. If these programs did real work, then the noise introduced by the real work would likely swamp out the differences that this paper seeks to measure.

The first abstract program ("Write to a global") was based heavily off of the program used in [\[P1640R0\]](#). The essentials of error handling were captured in that program, and there was little reason to change it significantly.

The second program ("Write through an argument") came about as part of investigations into the MSVC x64 performance anomaly where exceptions beat abort. This program demonstrates that minor changes, such as adding a parameter to a function, or using a smaller `mov` instruction, can affect the performance delta between exceptions and abort.

The third program ("Return a value") was authored to ensure that favoritism wasn't shown to the return code error handling strategy.

Each of the programs has a top level main, eight layers of "caller" functions, and a final "callee" function that decides whether to signal an error based off of a parameter. Eight was chosen mostly arbitrarily. The author wasn't able to get sufficient nop-randomization with a single "callee" function. Calling sixteen functions exceeded the size of the return stack buffer on the test machine.

§ 4. Starter test case for on-line analysis

In the code examples, anything using a `$` is something that gets generated before the C++ compiler or preprocessor ever sees it. `X`, `Y`, `Z`, and `K[0]` through `K[7]` are all random numbers in the range of [0,31], determined at build time.

To start with, the following code will be used:

```

struct Dtor$N {
    extern ~Dtor$N() {
        /* K[N] NOPs */
    }
};

struct inline_nops {
    /* only one inline_nops class per executable, no $ variables needed */
    inline inline_nops() {
        /* Z NOPs */
    }
    inline ~inline_nops() {
        /* 31 - Z NOPs */
    }
};

int global_int = 0;
int callee(bool do_err) {
    inline_nops n;
    if(do_err)
        return 1;
    return 0;
}

int caller$N(bool do_err) {
    /* 31 - K[N] NOPs */
    Dtor$N d;
    $if N == 7
        int e = callee(do_err);
    $else
        int e = caller$N+1(do_err);
    if(e) return e;
    global_int = 0;
    return 0;
}

int main() {
    /* Test happy path */
    startTimer();
    /* X NOPs */
    for (int i = 0; i < ITERATIONS; ++i) {
        /* invokes caller$N with N == 0 */
        (void)caller0(false);
    }
    /* 31 - X NOPs */
    endTimer();
    print(duration() / ITERATIONS);

    /* Test sad path */
    startTimer();
    /* Y NOPs */
    for (int i = 0; i < ITERATIONS; ++i) {
        (void)caller0(true);
    }
    /* 31 - Y NOPs */
    endTimer();
    print(duration() / ITERATIONS);
}

```

This code has some important properties.

- `callee()` will raise an error based off of runtime state
- `caller$N()` needs to clean up the `d` object in error and non-error conditions.
- `caller$N()` should only set `global_int` in success cases.
- The inserted NOPs allow for sampling various alignments while keeping the same program structure.

In the actual tests all the function bodies are in separate .cpp files, and link-time / whole-program optimizations aren't used. If they had been used, the entire program would get optimized away, removing our ability to measure error handling differences.

The initial part of this paper is only looking at three types of error handling.

- `abort`: The program directly calls `std::abort` when an error condition is encountered. No error handling code beyond `std::abort` is present in the code. No "sad" path benchmarks were taken for this case.
- `return_val`: Return an integer error code, where zero represents success.
- `throw_exception`: Throw an exception deriving from `std::exception`, that contains only an int. This should represent typical exception usage cases. The exception is only caught with a `catch(...)`.

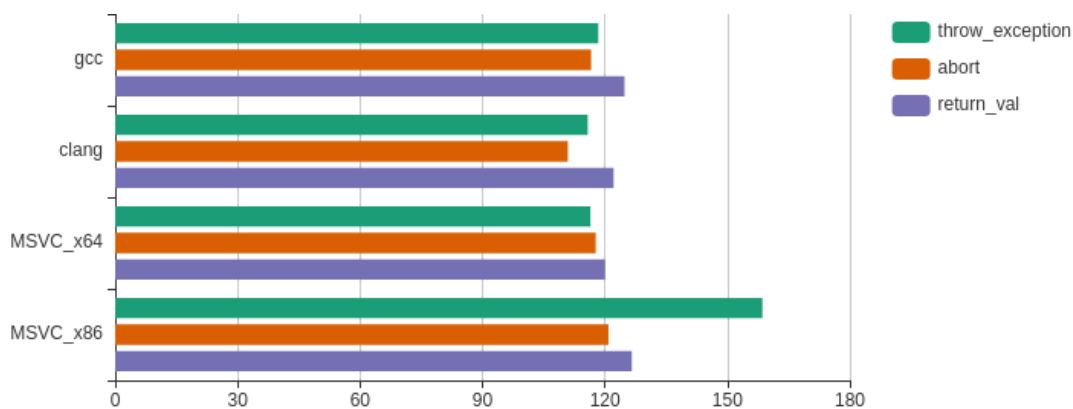
The actual code used for the benchmark can be found [on github](#).

§ 5. Happy path measurements

Abort and x64 exceptions have similar happy path performance characteristics. Return codes cost one to four extra cycles per function. x86 exceptions cost five to seven extra cycles per function in comparison to abort.

§ 5.1. Write to a global

Write to global: Median cycles per iteration. A histogram can be found [here](#)



§ 5.2. Write through an argument

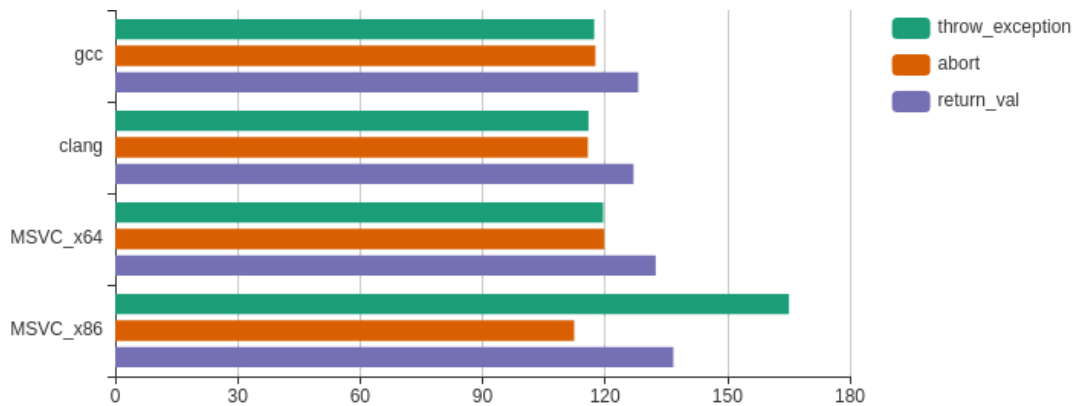
Instead of writing to a global, the code writes through a pointer argument.

```

int callee(int *val, bool do_err) {
    (void)val;
    inline_nops n;
    if(do_err)
        return 1;
    return 0;
}
int caller$N(int *val, bool do_err) {
    /* 31 - K[N] NOPs */
    Dtor$N d;
    $if N == 7
        int e = callee(val, do_err);
    $else
        int e = caller$N+1(val, do_err);
    if(e) return e;
    *val = 0;
    return 0;
}

```

Write through argument: Median cycles per iteration. A histogram can be found [here](#)



§ 5.3. "Return" a value

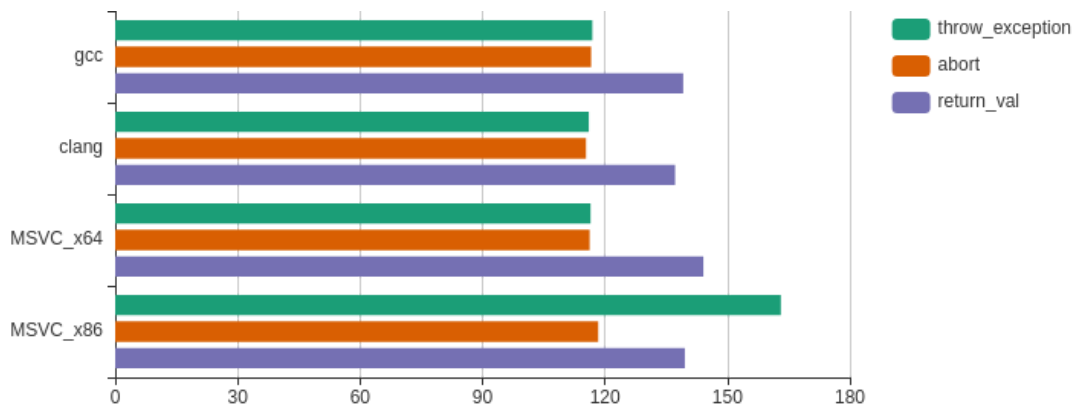
In this case, the code will return an incremented channel in whichever output channel is available. An output parameter is used when handling errors in an error code fashion.

```

int callee(int *val, bool do_err) {
    inline_nops n;
    if(do_err)
        return 1;
    *val = 0;
    return 0;
}
int caller$N(int *val, bool do_err) {
    /* 31 - K[N] NOPs */
    Dtor$N d;
    int out_val = 0;
    $if N == 7
        int e = callee(&out_val, do_err);
    $else
        int e = caller$N+1(&out_val, do_err);
    if(e) return e;
    *val = out_val + 1;
    return 0;
}

```

Return a value: Median cycles per iteration. A histogram can be found [here](#)



§ 6. Sad path measurements

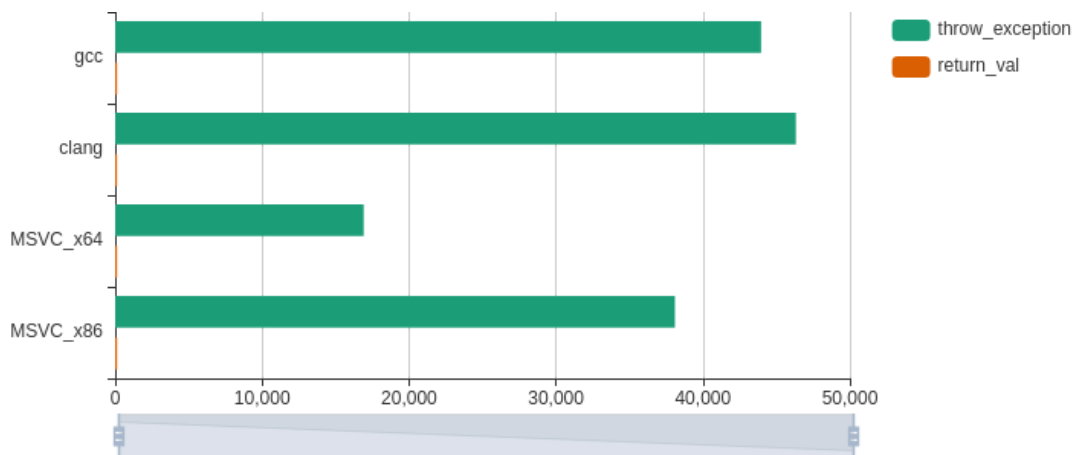
The following sad path measurements are best cases for the sad path. The sad path code is all hot. All the branches should predict as well as they can. On the exception front, `catch(...)` is used, so little-to-no RTTI-like matching code needs to run.

Even given this best case scenario, throwing an exception is more than 100x slower than returning an error code. The standard deviation of the exception runs is also more than 100x larger than returning an error code. This means that the determinism of throwing an exception is substantially worse than error codes. This is before throwing in complications like difficult catch block matching, concurrent execution fighting over dynamic allocation locks, or cold cache effects, all of which would likely make return codes look even better.

Sad path timings for abort aren't shown, as the recovery path is very system specific.

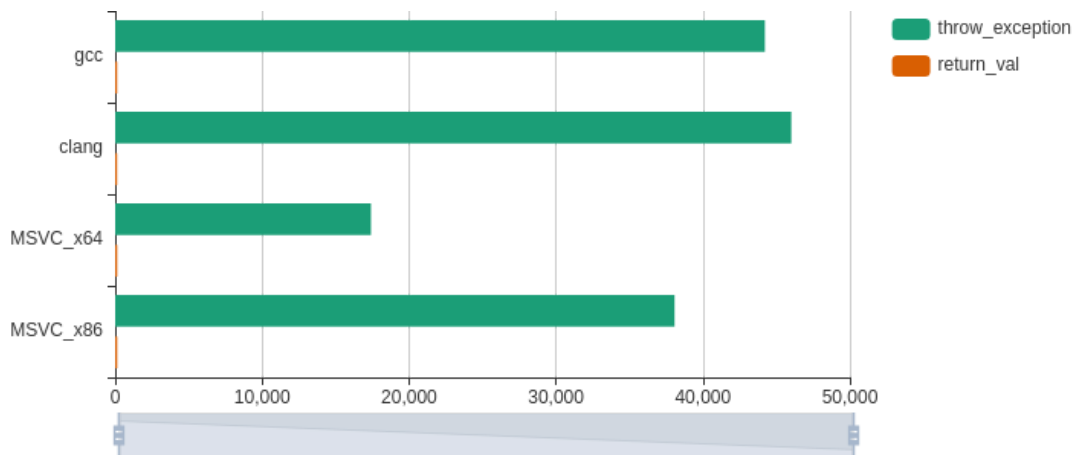
§ 6.1. Write to a global

Write to a global, sad path: Median cycles per iteration. A histogram can be found [here](#)



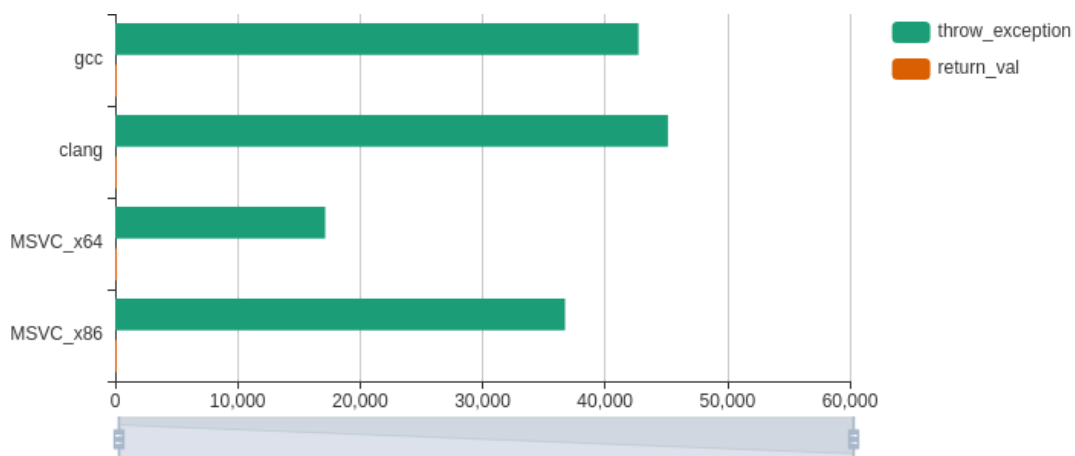
§ 6.2. Write through an argument

Write through an argument, sad path: Median cycles per iteration. A histogram can be found [here](#)



§ 6.3. "Return" a value

Return a value, sad path: Median cycles per iteration. A histogram can be found [here](#)



§ 7. Off-line analysis

The following analysis is done by examining assembly, but not running it. This analysis is valuable, but is no substitute for data on real machines.

§ 7.1. LLVM-MCA happy path off-line analysis

LLVM-MCA [\[MCA\]](#) is a tool that will analyze assembly and provide statistics on a hypothetical execution on a specific simulated CPU. This can be used to provide a hardware independent analysis of generated code. There is at least one substantial limitation though, in that calls and returns are typically not modeled. Regardless, the analysis is still useful.

This analysis focuses on the difference between exception builds and abort / no exception builds, all in an error neutral function. Only 64-bit x86-64 (Clang, GCC, and MSVC) and 64-bit ARM (Clang and GCC) code will be discussed. Exceptions on 32-bit x86 on Windows is so much slower in the happy path that it isn't interesting to dive in at the assembly level.

In general, there are some minor missed optimization opportunities in some of the smallest error handling examples.

The following code snippet will be analyzed:

```
struct Dtor {~Dtor();};
void callee();
int global;
void caller() {
    Dtor d;
    callee();
    global = 0;
}
```

Compiler	Exceptions	No exceptions
MSVC x64	<pre>sub rsp, 40 call void callee(void) // Note the ordering difference mov DWORD PTR int global, 0 lea rcx, QWORD PTR d\$[rsp] call Dtor::~~Dtor(void) add rsp, 40 ret 0</pre>	<pre>sub rsp, 40 call void callee(void) // Note the ordering difference lea rcx, QWORD PTR d\$[rsp] mov DWORD PTR int global, 0 call Dtor::~~Dtor(void) add rsp, 40 ret 0</pre>

With MSVC x64, the codegen is almost identical between code with exceptions and without. On MSVC x64, there are some optimization passes that don't run when exceptions are turned on, and one of those passes is responsible for switching the order of the `lea` and `mov` instructions. The benchmarks show a 1-3 cycle difference just from switching the order of those two instructions. This results in the exception based code being slightly faster than the abort based code for this use case. The "optimization" pass in the no-exception code causes a minor speed penalty.

Compiler	Exceptions	No exceptions
Clang 9.0 x64	1397 cycles over 100 iterations. Counts on Intel Ivy Bridge micro-architecture	1199 cycles over 100 iterations. Counts on Intel Ivy Bridge micro-architecture

	<pre> pushq %rbx subq \$16, %rsp callq callee() movl \$0, global(%rip) leaq 8(%rsp), %rdi callq Dtor::~Dtor() addq \$16, %rsp popq %rbx retq </pre>	<pre> pushq %rax // no subq here callq callee() movl \$0, global(%rip) movq %rsp, %rdi // movq instead of leaq callq Dtor::~Dtor() // no addq here popq %rax retq </pre>
GCC 8.2 x64	1397 cycles over 100 iterations. Counts on Intel Ivy Bridge micro-architecture	752 cycles over 100 iterations. Counts on Intel Ivy Bridge micro-architecture
	<pre> pushq %rbx subq \$16, %rsp call callee() // ordering difference movl \$0, global(%rip) leaq 15(%rsp), %rdi call Dtor::~Dtor() addq \$16, %rsp popq %rbx ret </pre>	<pre> // no pushq subq \$24, %rsp call callee() // ordering difference leaq 15(%rsp), %rdi movl \$0, global(%rip) call Dtor::~Dtor() addq \$24, %rsp // no popq ret </pre>

x64 GCC and Clang both generate slower code for this use case when exceptions are enabled. This is a missed optimization, but isn't inherent to the design of exceptions.

In the timings, GCC's abort code was 2 cycles faster than GCC's exception code. Clang's abort code was 5 cycles faster than Clang's exception code.

Compiler	Exceptions	No exceptions
Clang 9.0 ARM64	684 cycles over 100 iterations. Counts on exynos-m5 architecture	665 cycles over 100 iterations. Counts on exynos-m5 architecture
	<pre> str x19, [sp, #-32]! stp x29, x30, [sp, #16] add x29, sp, #16 bl callee() adrp x8, global add x0, sp, #8 str wzr, [x8, :lo12:global] bl Dtor::~Dtor() ldp x29, x30, [sp, #16] ldr x19, [sp], #32 ret </pre>	<pre> sub sp, sp, #32 // sub instead of str stp x29, x30, [sp, #16] add x29, sp, #16 bl callee() adrp x8, global add x0, sp, #8 str wzr, [x8, :lo12:global] bl Dtor::~Dtor() ldp x29, x30, [sp, #16] add sp, sp, #32 // add instead of ldr ret </pre>
GCC 8.2 ARM64	639 cycles over 100 iterations. Counts on exynos-m5 architecture	639 cycles over 100 iterations. Counts on exynos-m5 architecture

<pre> stp x29, x30, [sp, -48]! mov x29, sp bl callee() adrp x1, .LANCHOR0 add x0, sp, 40 str wzr, [x1, #:lo12:.LANCHOR0] bl Dtor::~Dtor() ldp x29, x30, [sp], 48 ret </pre>	<pre> stp x29, x30, [sp, -32]! // different constant mov x29, sp bl callee() adrp x1, .LANCHOR0 add x0, sp, 24 // different constant str wzr, [x1, #:lo12:.LANCHOR0] bl Dtor::~Dtor() ldp x29, x30, [sp], 32 // different constant ret </pre>
---	---

ARM64 GCC and Clang both generate different code for this use case when exceptions are enabled. This is likely another missed optimization. With GCC, the only penalty that is paid is slightly more stack usage.

§ 7.2. Sad path affecting the happy path and macro-benchmark theorizing

GCC and Clang x64 place an exception "landing pad" at the end of each function that needs to run user code. The assembly of the following code, including the landing pad, will be analyzed:

```

struct Dtor {~Dtor();};
void callee();
int global;
void caller() {
    Dtor d;
    callee();
    global = 0;
}

```

Clang 9.0	GCC 7.4	GCC 9.2
<pre> push rbx sub rsp, 16 call callee() mov dword ptr [rip + global], 0 lea rdi, [rsp + 8] call Dtor::~Dtor() add rsp, 16 pop rbx ret // Begin landing pad mov rbx, rax lea rdi, [rsp + 8] call Dtor::~Dtor() mov rdi, rbx call _Unwind_Resume </pre>	<pre> push rbx sub rsp, 16 call callee() lea rdi, [rsp+15] mov DWORD PTR global[rip], 0 call Dtor::~Dtor() add rsp, 16 pop rbx ret // Begin landing pad lea rdi, [rsp+15] mov rbx, rax call Dtor::~Dtor() mov rdi, rbx call _Unwind_Resume </pre>	<pre> push rbp sub rsp, 16 call callee() mov DWORD PTR global[rip], lea rdi, [rsp+15] call Dtor::~Dtor() add rsp, 16 pop rbp ret // Begin landing pad mov rbp, rax jmp .L2 caller() [clone .cold]: .L2: lea rdi, [rsp+15] call Dtor::~Dtor() mov rdi, rbp call _Unwind_Resume </pre>

In all three cases, some cold landing pad code is directly adjacent to hot happy path code. This doesn't cause any problems in microbenchmarks, but in theory, it could reduce instruction cache utilization in larger benchmarks. Starting in GCC 8.1, GCC will only put a tiny trampoline in the landing pad. The bulk of the landing pad code is moved into a different, cold location. This should, in theory, improve instruction cache utilization in the happy path compared to Clang and earlier GCCs, though it will still be at a minor disadvantage compared to terminate based implementations. The author has no

measurements to confirm or reject this theory though. [Spencer18] measured a small, non-zero performance degradation (<1%) in the happy path in a AAA game when turning on exceptions.

The MSVC x64 /d2FH4 implementation does not place any exception handling code near the happy path. The MSVC x86 sad path exception handling code is also far away from the happy path, but the happy path does exception bookkeeping.

§ 8. Conclusion

Exceptions are reasonable error handling strategies in many environments, but they don't serve all needs in all use cases. C++ needs high-performance, deterministic, standards-conforming ways to signal failures in constructors and operator overloads. Currently, exceptions do well on the happy path, but there is no good answer for the sad path. C++ is built on the foundation that you don't pay for what you don't use, and that you can't write the language abstractions better by hand. This paper provides evidence that you can write error handling code by hand that results in faster code on the sad path than the equivalent exception throwing code. In each of the sad path test cases, return values beat exceptions by more than a factor of 100.

§ 9. Acknowledgments

Niall Douglas, Herb Sutter, Nathan Sidwell, Joshua Cannon, Mike Petersen, Brad Keryan, and Ben Saks all provided valuable early feedback on this paper.

Charts generated with [ECharts].

§ Appendix A: Additional error handling types

While researching this paper, data was gathered on some additional error handling mechanisms. The data is provided here with a brief description of the error handling mechanism. Only a minimal amount of code and explanation will be provided though. This data could be useful in future error handling discussions.

In each of these cases, the struct involved is of the following form:

```
struct error_struct {
    void *error = nullptr;
    void *domain = nullptr;
};
```

A null error pointer indicates no error.

- ref_struct: Pass an `error_struct` by reference, and check `error` after each call.

```
void callee(bool do_err, error_struct &e)
```

- return_struct: Like return_val, but with an `error_struct` instead.

```
error_struct callee(bool do_err)
```

- return_nt_struct: Like return_struct, but `error_struct` has a destructor of the form `~error_struct() {}`. "nt" means "non-trivial".

```
error_struct /* with non-trivial destructor */ callee(bool do_err)
```

- expected_struct: This uses Sy Brand's [\[Expected\]](#) library with `error_struct` as the error type. This should closely resemble the proposed `std::expected`.

```
tl::expected<void, error_struct> callee(bool do_err)
```

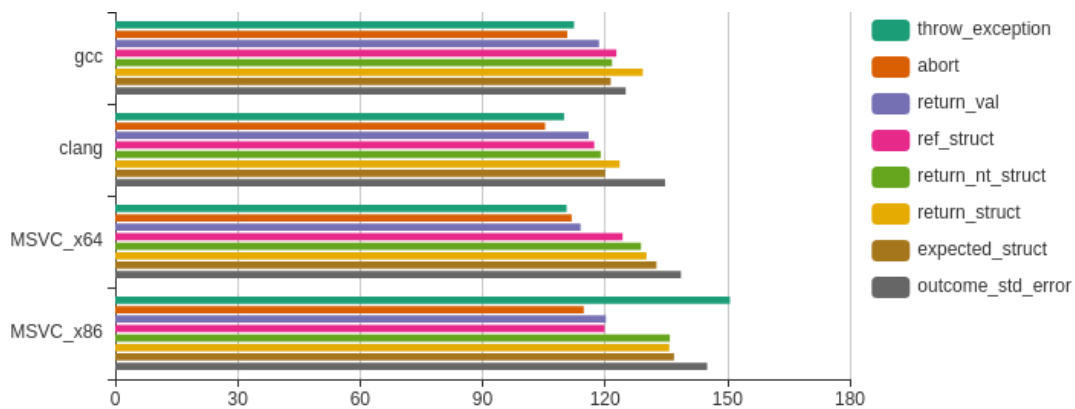
- outcome_std_error: This uses a branch of Niall Douglas's [\[Outcome\]](#) library. This branch is specially optimized for GCC on the happy path.

```
outcome::experimental::status_result<void> callee(bool do_err)
```

In the sad path graphs, exceptions are omitted so that the remaining values can be easily compared.

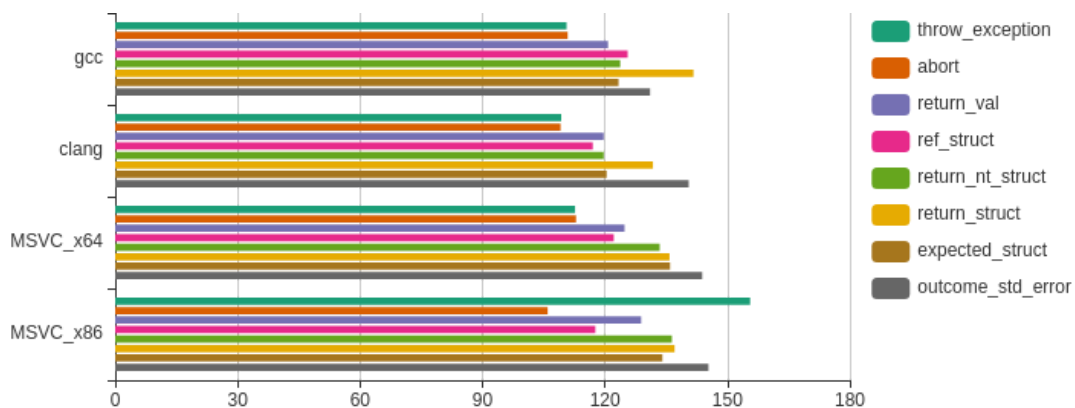
§ Write to a global (happy path)

Write to global: Median cycles per iteration. A histogram can be found [here](#)



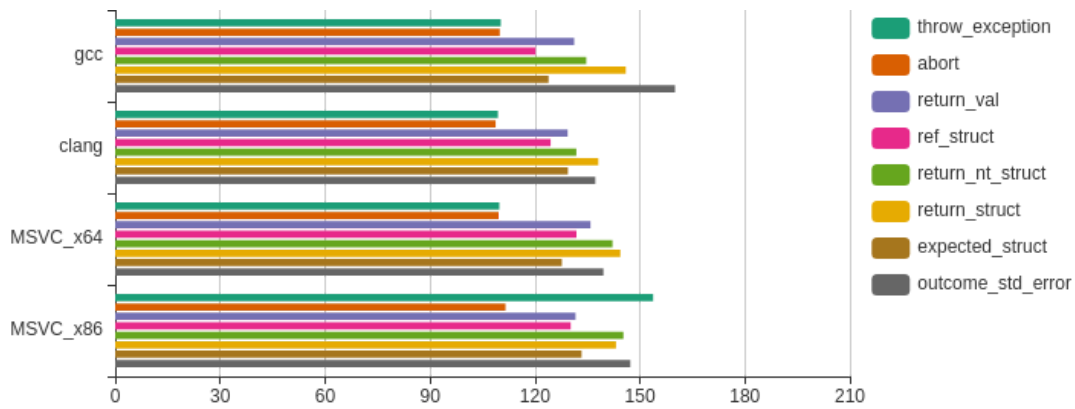
§ Happy path: Write through an argument

Write to global: Median cycles per iteration. A histogram can be found [here](#)



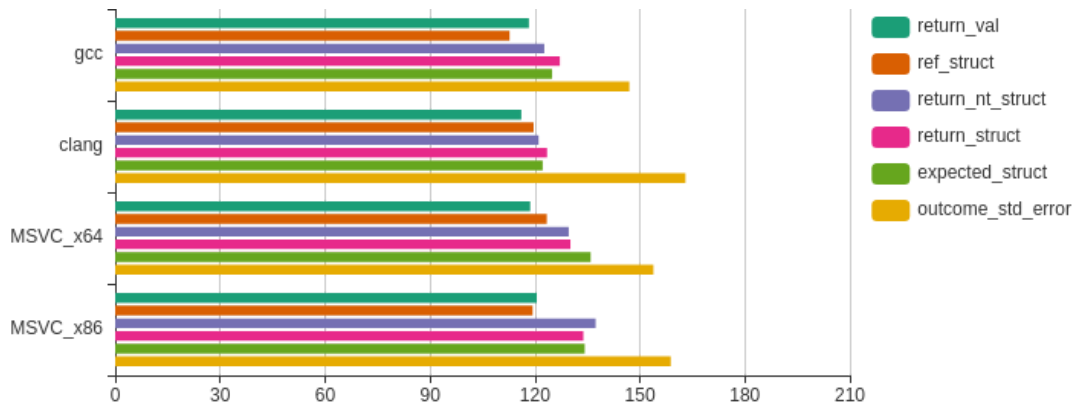
§ Happy path: "Return" a value

Write to global: Median cycles per iteration. A histogram can be found [here](#)



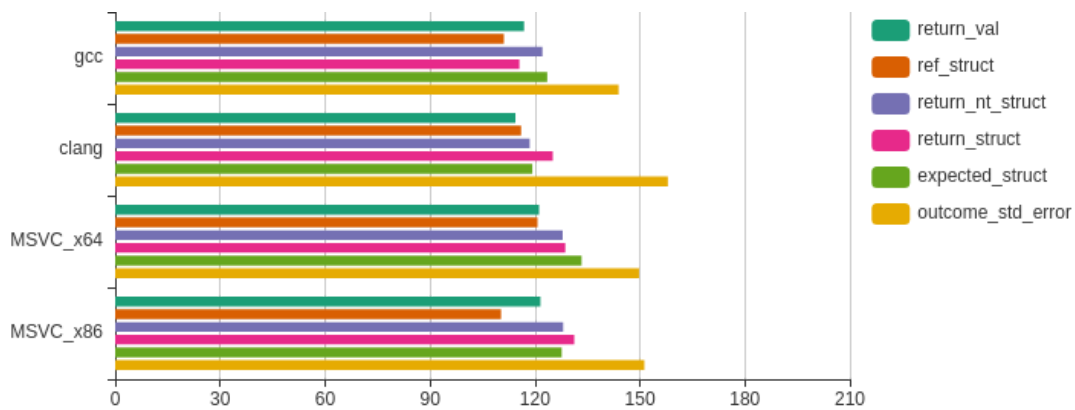
§ Sad path: Write to a global

Write to global: Median cycles per iteration. A histogram can be found [here](#)



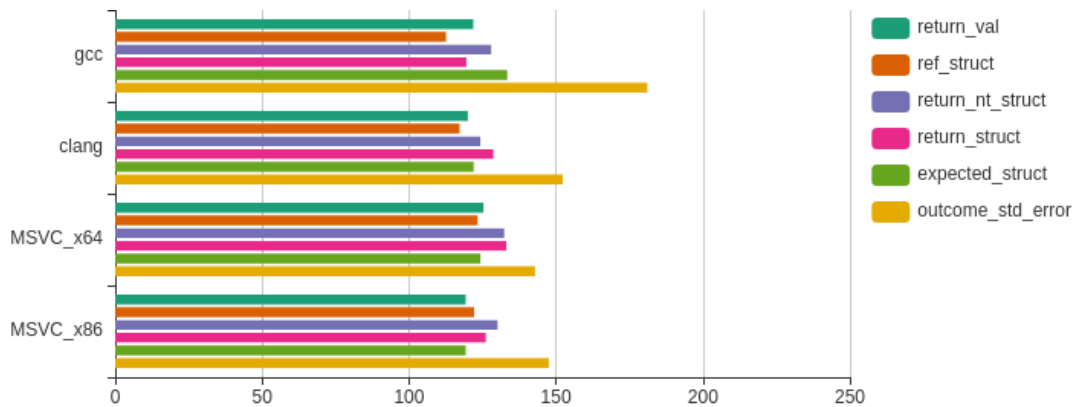
§ Sad path: Write through an argument

Write to global: Median cycles per iteration. A histogram can be found [here](#)



§ Sad path: "Return" a value

Write to global: Median cycles per iteration. A histogram can be found [here](#)



§ Appendix B: The build flags

§ MSVC

The compiler and flags are the same for 32-bit and 64-bit builds, except that the 32-bit linker uses /machine:x86 and the 64-bit linker uses /machine:x64

/d2FH4 is a critical flag for these benchmarks. Without it, the results are drastically different. See [\[MoFH4\]](#) for a description of this flag.

Compiler marketing version: Visual Studio 2019

Compiler toolkit version: 14.22.27905

cl.exe version: 19.22.27905

Compiler codegen flags (no exceptions): /GR- /Gy /Gw /O2 /MT /d2FH4 /std:c++latest /permissive- /DNDEBUG

Compiler codegen flags (with exceptions): /EHsc /GR /Gy /Gw /O2 /MT /d2FH4 /std:c++latest /permissive- /DNDEBUG

Linker flags: /OPT:REF /release /subsystem:CONSOLE /incremental:no /OPT:ICF /NXCOMPAT /DYNAMICBASE *.obj

§ Clang x64

Toolchains used:

- Clang 8.0.0 and libc++
- System linker from Ubuntu 14.04.3's GCC 4.8.4 installation

Compiler codegen flags (no exceptions): -fno-exceptions -fno-rtti -O2 -mtune=ivybridge -ffunction-sections -fdata-sections -std=c++17 -stdlib=libc++ -static -DNDEBUG

Compiler codegen flags (exceptions): -O2 -mtune=ivybridge -ffunction-sections -fdata-sections -std=c++17 -stdlib=libc++ -static -DNDEBUG

Linking flags: -Wl,--gc-sections -pthread -static -static-libgcc -stdlib=libc++ *.o libc++abi.a

§ GCC x64

Toolchain used: GCC 7.3.1 from the Red Hat Developer Toolset 7.1

Compiler codegen flags (no exceptions): -fno-exceptions -fno-rtti -O2 -mtune=ivybridge -ffunction-sections -fdata-sections -std=c++17 -static -DNDEBUG

Compiler codegen flags (exceptions): -O2 -mtune=ivybridge -ffunction-sections -fdata-sections -std=c++17 -static -DNDEBUG

Linking flags: -Wl,--gc-sections -pthread -static -static-libgcc -static-libstdc++ *.o

§ Appendix C: The code

As stated before, this isn't the exact code that was benchmarked. In the benchmarked code, functions were placed in distinct translation units in order to avoid inlining. The following code is provided to demonstrate what the error handling code looks like.

§ Common support code

► Expand to see code snippets

§ Write to a global

► Expand to see code snippets

§ Write through an argument

► Expand to see code snippets

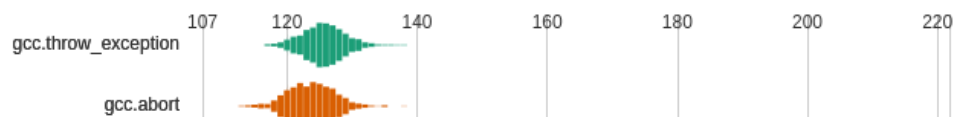
§ "Return" a value

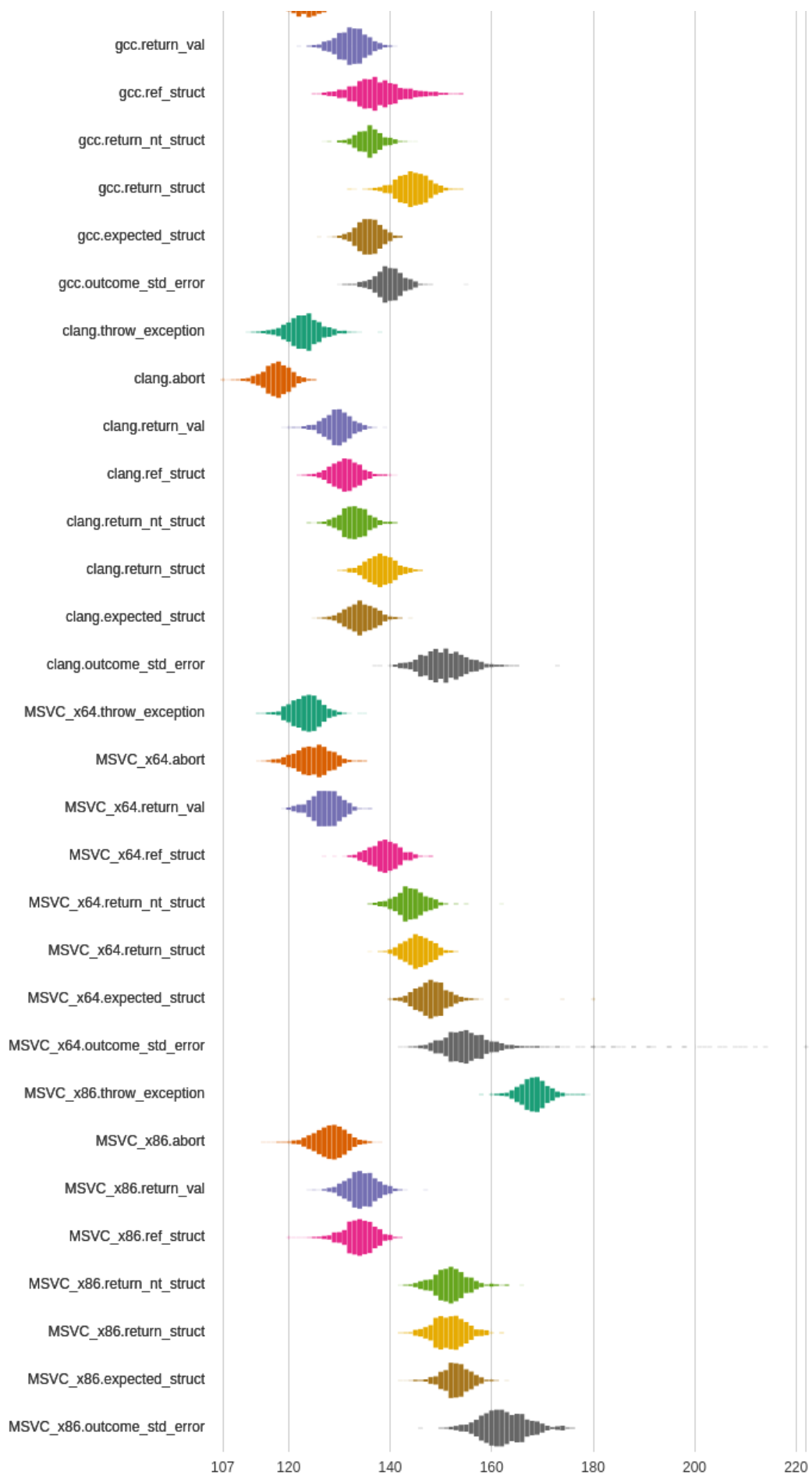
► Expand to see code snippets

§ Appendix D: Histograms

§ Happy path: Write to a global

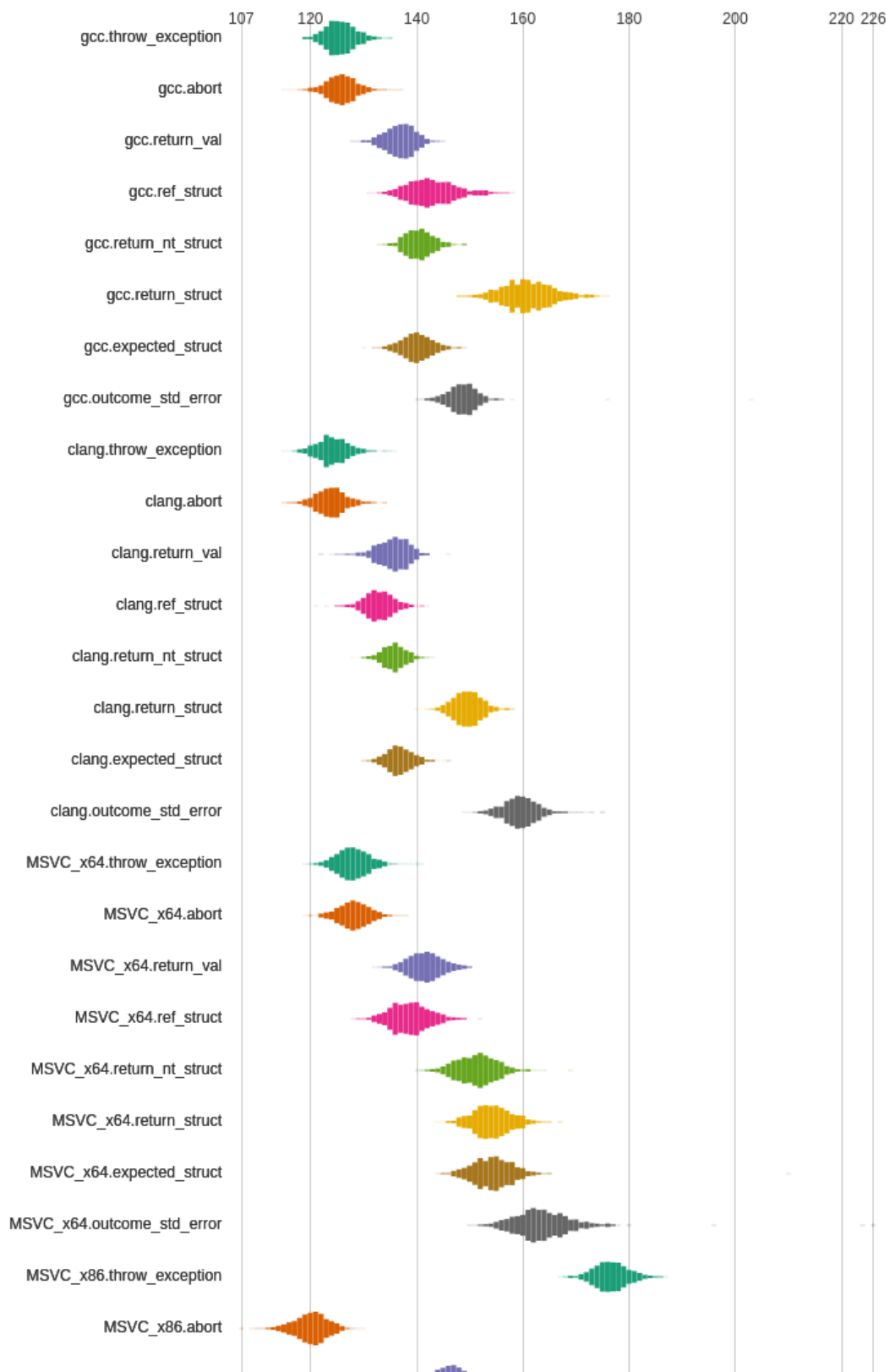
Write to global: Cycles per iteration histogram

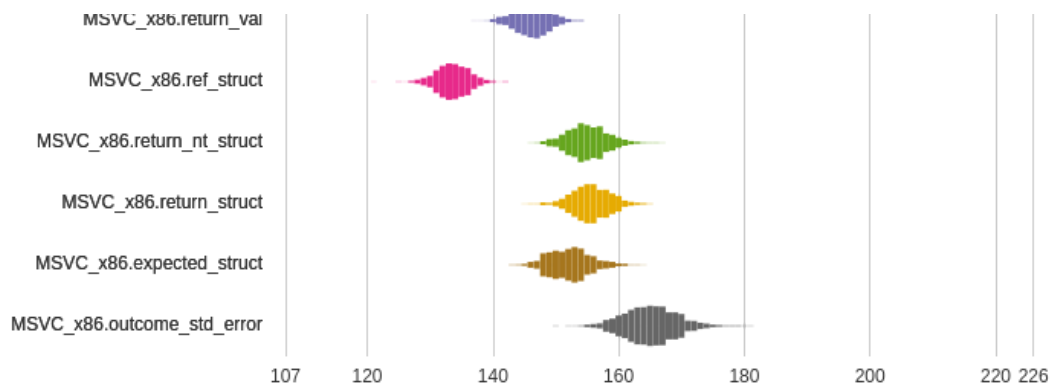




§ Happy path: Write through an argument

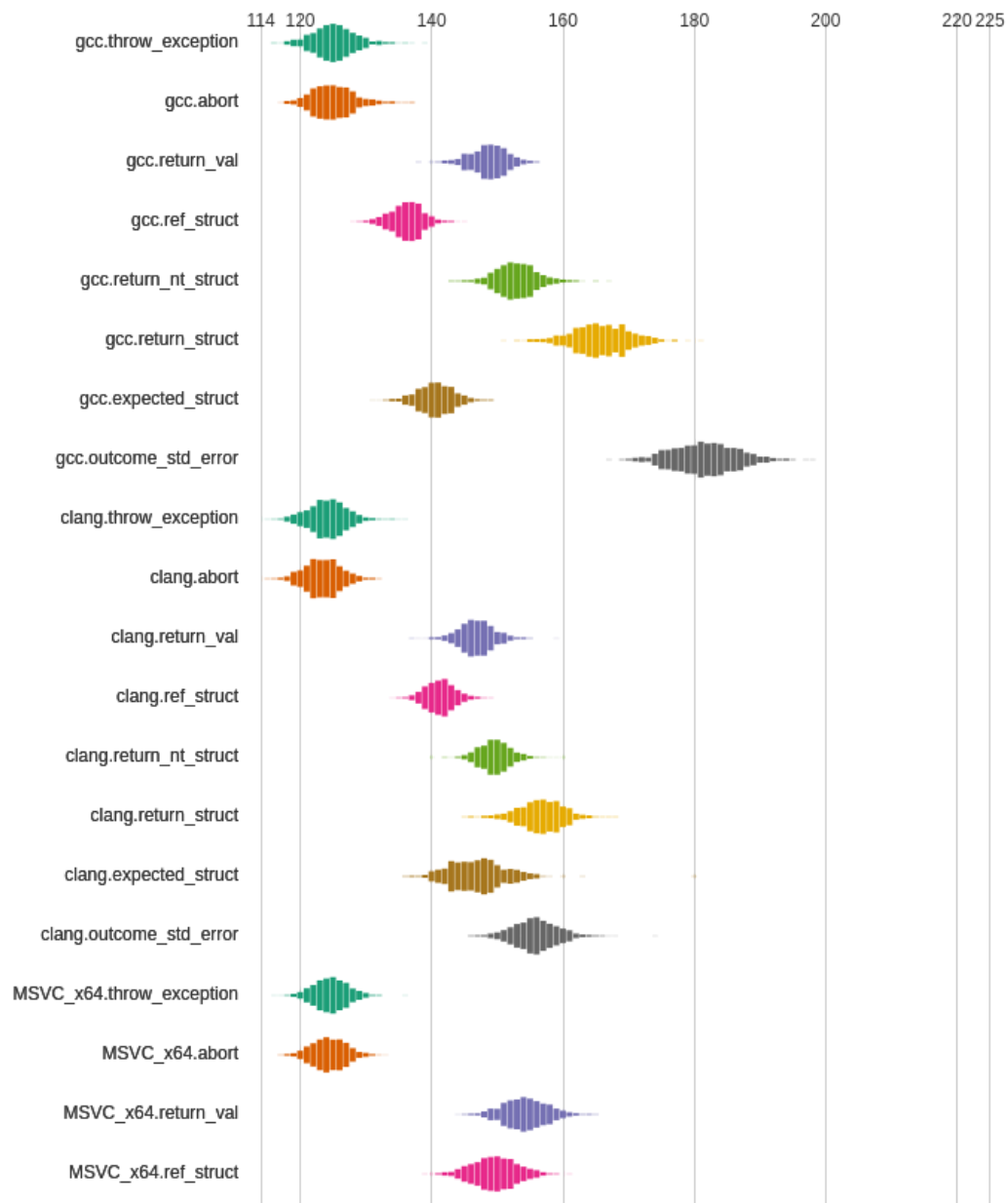
Write through an argument: Cycles per iteration histogram

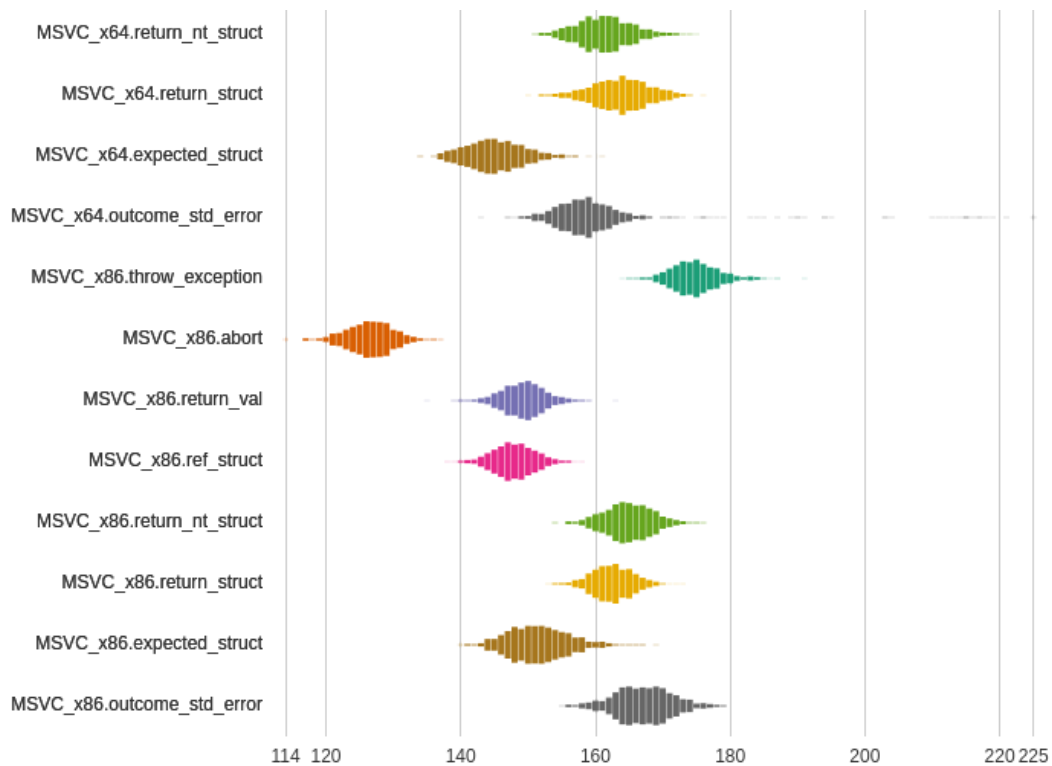




§ Happy path: "Return" a value

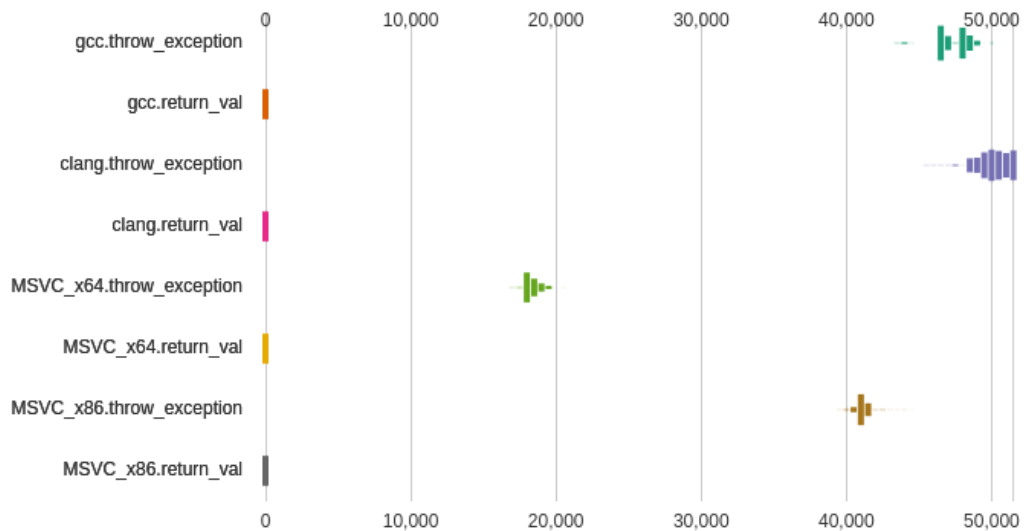
Return a value: Cycles per iteration histogram





§ Exception sad path: Write to a global

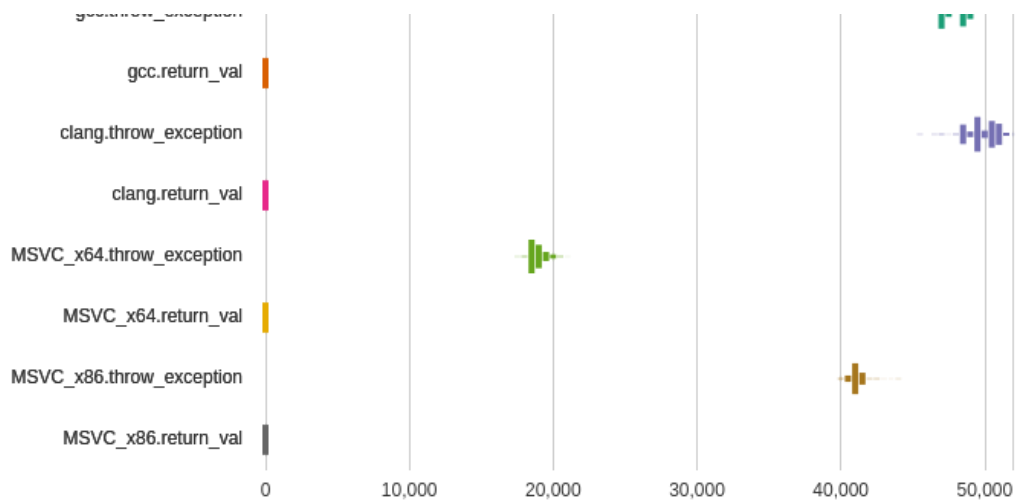
Write to a global: Cycles per iteration histogram. throw_exception and return_val only.



§ Exception sad path: Write through an argument

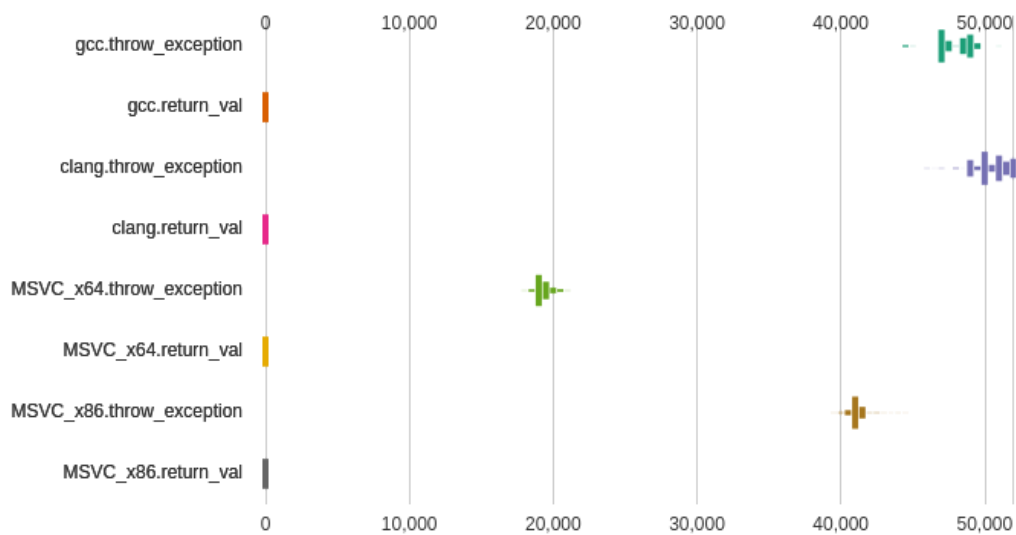
Write through an argument: Cycles per iteration histogram. throw_exception and return_val only.





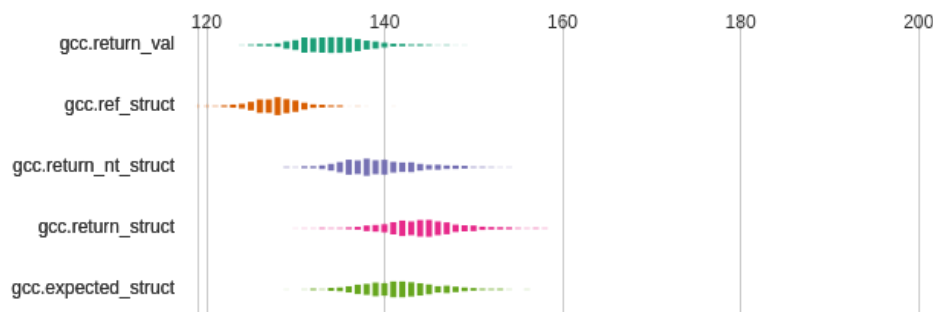
§ Exception sad path: "Return" a value

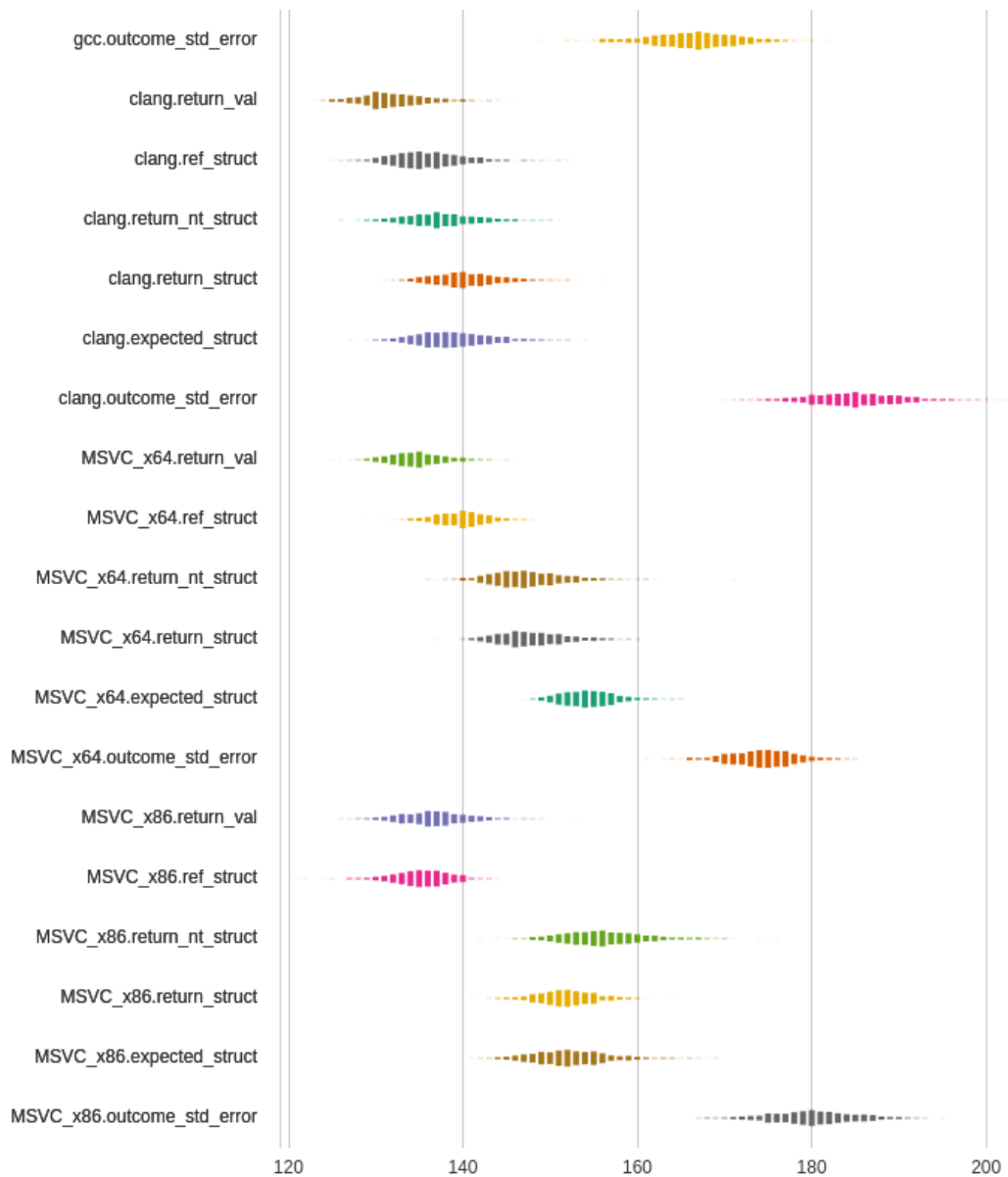
Return a value: Cycles per iteration histogram. throw_exception and return_val only.



§ Other sad path: Write to a global

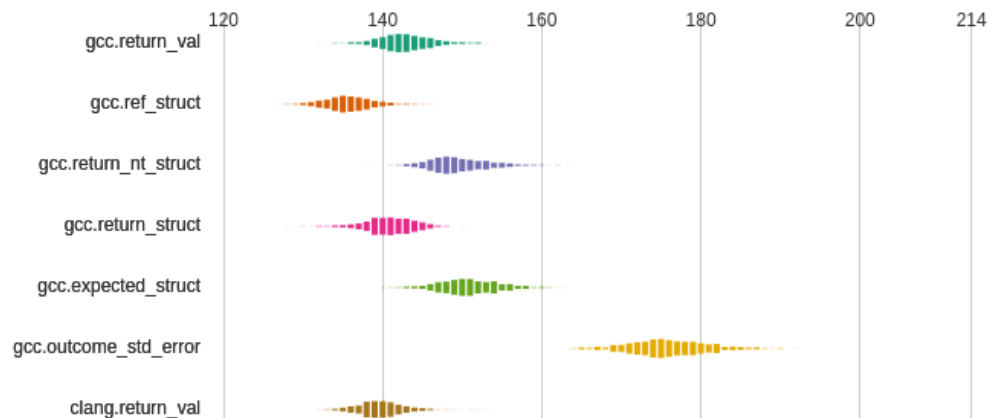
Write to a global: Cycles per iteration histogram. throw_exception omitted.

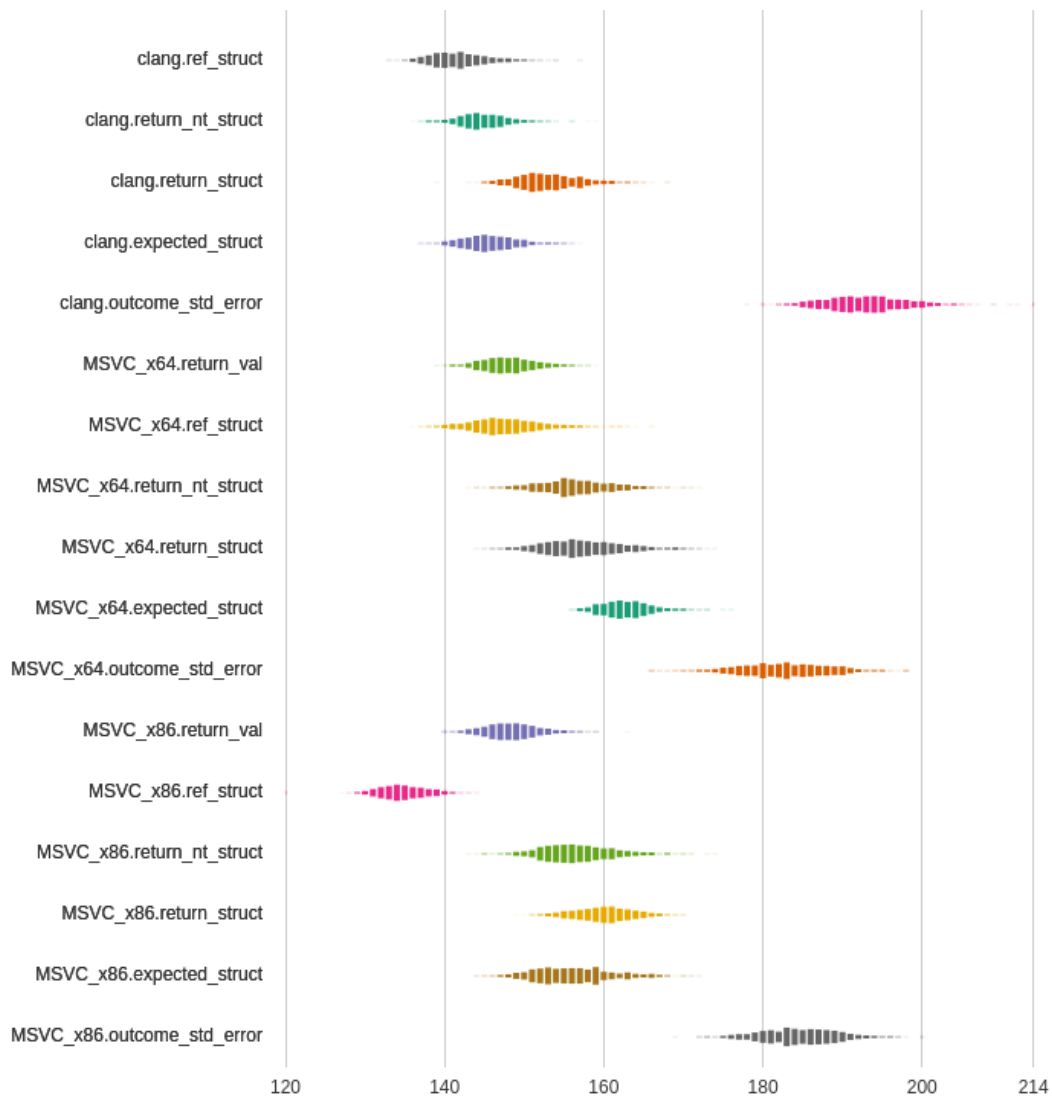




§ Other sad path: Write through an argument

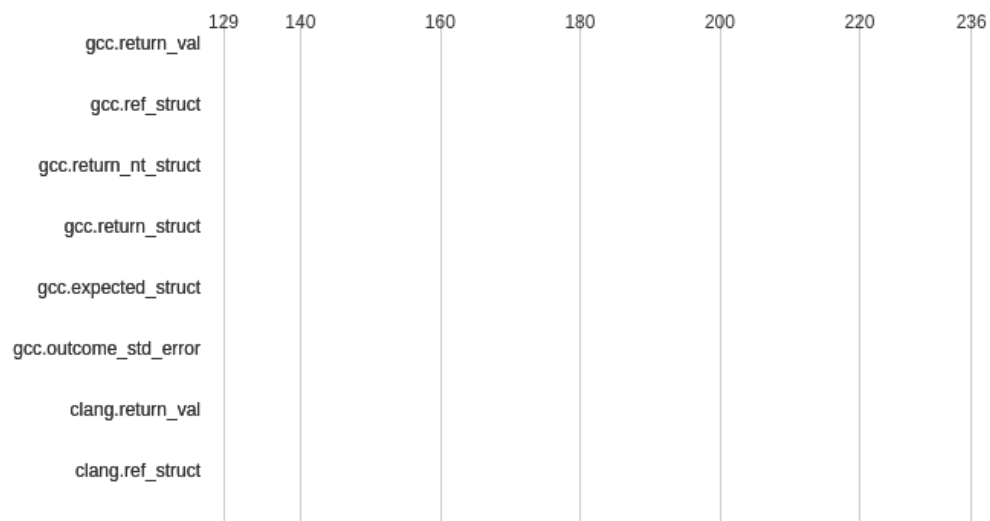
Write through an argument: Cycles per iteration histogram. throw_exception omitted.

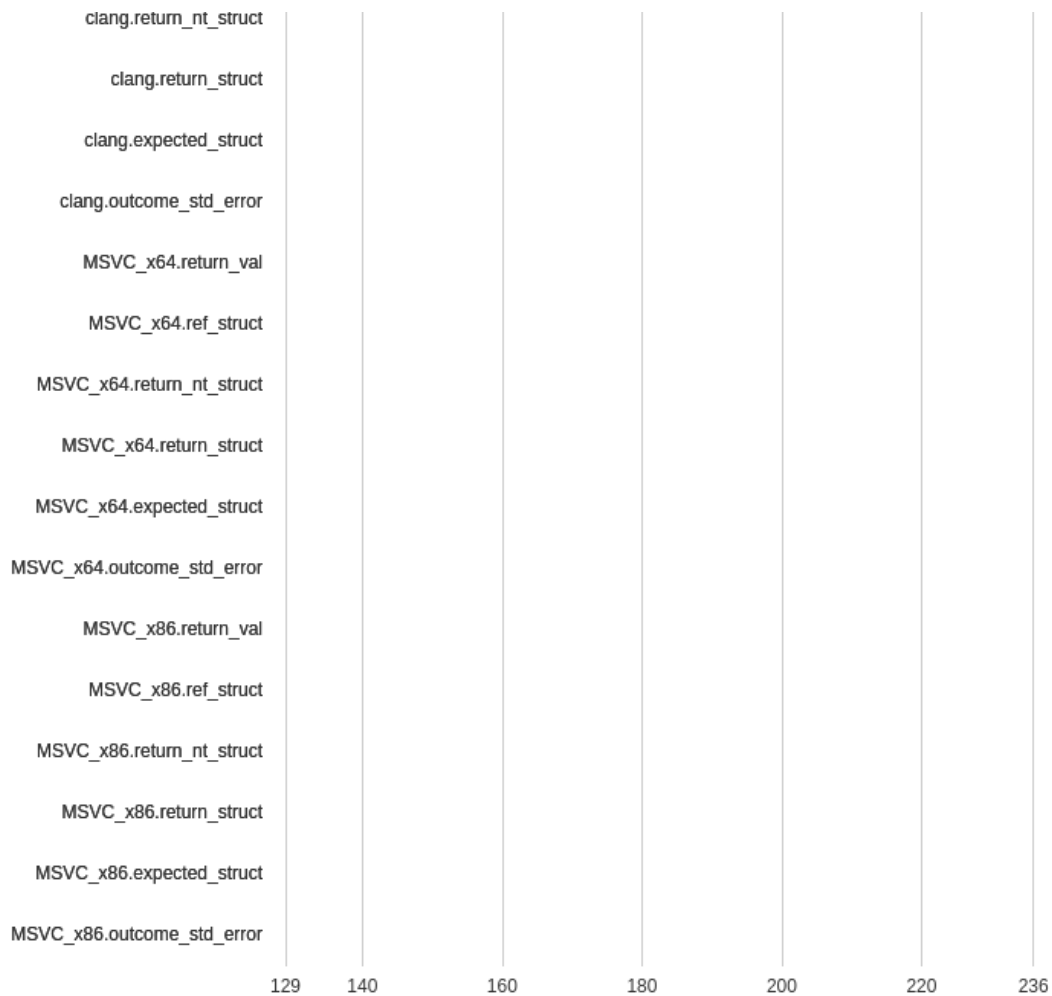




§ Other sad path: "Return" a value

Return a value: Cycles per iteration histogram. throw_exception omitted.





§ References

§ Informative References

[Bakhvalov]

Denis Bakhvalov. Code alignment issues. URL: https://dendibakh.github.io/blog/2018/01/18/Code_alignment_issues

[ECharts]

ECharts. URL: <https://echarts.apache.org/en/index.html>

[Expected]

Sy Brand. expected: C++11/14/17 std::expected with functional-style extensions. URL: <https://github.com/TartanLlama/expected>

[MCA]

llvm-mca - LLVM Machine Code Analyzer. URL: <https://llvm.org/docs/CommandGuide/llvm-mca.html>

[MoFH4]

Modi Mo. Making C++ Exception Handling Smaller On x64. URL: <https://devblogs.microsoft.com/cppblog/making-cpp-exception-handling-smaller-x64/>

[Outcome]

Niall Douglas. outcome: Provides very lightweight outcome<T> and result<T> (non-Boost edition). URL: <https://github.com/ned14/outcome>

[P1640R0]

Ben Craig. [Error size benchmarking](https://wg21.link/p1640r0). 3 June 2019. URL: <https://wg21.link/p1640r0>

[Spencer18]

Michael Spencer. [C++Now 2018: How Compilers Reason About Exceptions](https://www.youtube.com/watch?v=C4gpj-MDstY). URL: <https://www.youtube.com/watch?v=C4gpj-MDstY>